COMP 2002 Assignment 2 — group guide

Group 100: Adib Pathan, Oliver Stachowiak, and Zhouyi Tang.

Shared scaffold for the parallel mergesort. Each person writes one piece. Every person can explain all three. The mark is a group mark.

- Due: Friday 30 October 2026, 11:59pm
- Weight: 30% of the course
- Late submissions are not graded
- Code that does not compile on Gradescope scores zero
- The oral is already booked. All three attend, with a webcam and ID. The tutor may ask any member.

This file stays at the repository root. It is not part of the Gradescope folder.

Marks

Criterion	Points	High Distinction starts above
Technical functionality (Gradescope)	48	40.7
Code quality and design	20	16.9
Oral: concurrency knowledge	16	13.5
Oral: code comprehension	16	13.5

Who writes what

A column is finished when the other two can explain it with the code on screen and the author stays quiet.

Member	Owns	Handoff	Teaches the other two
Zhouyi	`merge` and `my_mergesort` in `mergesort.c`	Cutoff 0 sorts n=2, duplicates, already sorted, reversed, and a hand-traced 8-element array	Inclusive split, why `B` exists, and how the merged run returns to `A`
Adib	`buildArgs` and `parallel_mergesort` in `mergesort.c`	Cutoff 0, 1, and 2 build the thread tree below. Both children are joined. Each child argument is freed once	`pthread_create` is the recursive step, and join-before-merge keeps `A` and `B` safe
Oliver	Test matrix, timings, README, Gradescope	The cases below, a timing table for 100,000,000 elements at levels 0–8, README from the template	Speedup of at least 2× against level 0, and why times flatten near level 5

Adib and Zhouyi can swap columns if the group prefers. Oliver stays on verification. The oral still covers every column for every member.

Shared contract

The three pieces meet on these rules. Indexes are inclusive. Level 0 is `main`, covering `0` through `n-1`. There is no tree object. The threads are the tree.

Serial range (Zhouyi)

`my_mergesort(left, right)` sorts one inclusive range on the calling thread. When `left` equals `right`, that one element is sorted. Otherwise the range splits at `mid = (left + right) / 2` into `[left, mid]` and `[mid + 1, right]`. Both halves are sorted, then merged.

`merge(left, mid, mid + 1, right)` reads two adjacent sorted runs and writes the merged run into the same indexes of `B`, then copies that run back into `A`. A thread touches only `A[left..right]` and `B[left..right]`.

Threaded range (Adib)

`buildArgs(left, right, level)` allocates one `struct argument`. `pthread_create` can pass only one `void *`, so the struct carries `left`, `right`, and `level`. The new thread starts in `parallel_mergesort`.

When `level` has reached `cutoff`, the thread calls `my_mergesort` and creates no children. Under the cutoff it creates two children, joins both, then merges. `main` frees the level-0 argument. The parent frees the two child arguments after the joins. The thread leaves its incoming argument for the caller to free.

Thread tree

Cutoff	What runs	Who checks it
0	Level 0 is already at the cutoff. `main` sorts the whole array with `my_mergesort`. No extra threads.	Zhouyi, then Adib and Oliver
1	`main` creates two threads. Each sorts one half serially. `main` joins both, then merges.	Adib, then Zhouyi and Oliver
2	Each level-1 thread creates two children, joins them, and merges, before `main` merges the two halves.	Adib, then Zhouyi and Oliver
5 and above	Times should keep improving until about level 5, then flatten. Target is at least 2× faster than cutoff 0 at n = 100,000,000.	Oliver, then Adib and Zhouyi

Order of work

When	Whole group	Owner
16–19 Oct	Trace an 8-element sort on paper. Review the serial code line by line.	Zhouyi: cutoff 0
20–23 Oct	Draw the thread tree for cutoff 0, 1, and 2. Review create, join, and free against the contract above.	Adib: parallel sort
24–27 Oct	Run the cases and the 100,000,000-element timings. Check range overlaps, leaks, and double frees. Submit to Gradescope. Resubmits are allowed before the deadline.	Oliver: tests, README, submission
28–29 Oct	Mock oral. Each member answers the questions below with the code on screen. The author stays quiet.	Rotate the questioner
30 Oct, before 11:59pm	Final Gradescope submit. Code that does not compile scores zero.	Oliver confirms the folder matches the starter layout

Questions every member can answer

A High Distinction answer names the mechanism and what goes wrong if it is skipped.

Question	Answer the group should be able to give
How does `parallel_mergesort` recur?	`pthread_create` starts the new thread in `parallel_mergesort`. That call is the recursive step. A direct call would stay on the same thread.
What changes from cutoff 0 to cutoff 2?	0 is one serial sort. 1 is two serial halves plus one merge in `main`. 2 adds a split inside each half. Draw the nodes.
Why join before merging?	The parent range covers both children. Merging early races on `A` and `B`. `pthread_join` is the wait. Disjoint ranges plus that wait are the synchronisation.
Where may two threads write?	Only inside their own inclusive index range, in both `A` and `B`. Overlapping ranges corrupt the sort.
Who allocates and frees each argument?	`buildArgs` allocates. The parent frees the two child structs after join. `main` frees the root. Freeing the incoming pointer inside the thread double-frees the root.
Why does speedup stop improving?	Past a handful of levels, core count, merge cost, and thread-creation cost dominate. More threads stop adding speed.

Submission rules

Write the sort in `mergesort.c`. Leave `mergesort.h` unchanged. Submit the starter folder layout, including the Makefile. Fill the README from `README.template`, including a Sources section for anything used outside the course materials.